A Project Report on

LawGrid : AI Legal Assistant For Antology-Driven Legal Reasoning And Support

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering - Data Science

by

Priyal Madvi (22107020)
Sneha Gupta (22107042)
Ayush Gupta (23207007)
Rushikesh Palekar(23207004)

Under the Guidance of

Dr. Pravin Adivarekar
Prof. Harsha Zope

Department of Computer Science and Engineering - Data Science
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2025-2026

Approval Sheet

This Project Report entitled ***"LawGrid:AI Legal Assistant For Antology- Driven Legal Reasoning And Support "*** Submitted by ***"Priyal Madvi "(22107020),"Sneha Gupta "(22107042),"Ayush Gupta "(23207007),"Rushikesh Palekar"(23207004)*** is approved for the partial fulfillment of the requirement for the award of the degree of ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science*** from ***University of Mumbai***.

Prof. Harsha Zope
Co-Guide

Dr. Pravin Adivarekar
Guide

Dr. Pravin Adivarekar
HOD, Computer Science and Engineering - Data Science

Place:A.P.Shah Institute of Technology, Thane
Date:

CERTIFICATE

This is to certify that the project entitled ***"LawGrid :AI Legal Assistant For Antology-Driven Legal Reasoning And Support"*** submitted by ***"Priyal Madvi" (2107020),"Sneha Gupta" (22107042),"Ayush Gupta" (23207007),"Rushikesh Palekar" (23207004)*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science***,to the University of Mumbai,is a bonafide work carried out during academic year 2025-2026.

Prof. Harsha Zope
Co-Guide

Dr. Pravin Adivarekar
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place:A.P.Shah Institute of Technology, Thane
Date:

Acknowledgement

We have great pleasure in presenting the report on **LawGrid : AI Legal Assistant For Antology-Driven Legal Reasoning And Support.** We take this opportunity to express our sincere thanks towards our guide **Dr. Pravin Adivarekar** & Co-Guide **Prof. Harsha Zope** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Dr. Pravin Adivarekar** Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Priyal Madvi
(22107020)

Sneha Gupta
(22107042)

Ayush Gupta
(23207007)

Rushikesh Palekar
(23207004)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Priyal Madvi(22107020)

(Signature)
Sneha Gupta (22107042)

(Signature)
Ayush Gupta(23207007)

(Signature)
Rushikesh Palekar (23207004)

Date:

Abstract

The AI Legal Assistant project is an advanced, all-in-one platform that leverages artificial intelligence, natural language processing (NLP), and conversational technology to transform how individuals and businesses handle legal tasks. It enables users to generate legally compliant contracts and documents through guided, automated drafting, minimizing errors and ensuring regulatory adherence. When users upload existing legal documents, the system uses AI-driven risk detection to identify potential issues such as ambiguous clauses, compliance gaps, or unfavorable terms, and then suggests actionable solutions. Its built-in chatbot functions as a virtual legal advisor, capable of answering queries, explaining complex legal language in plain terms, and providing step-by-step guidance in real time. Additionally, the platform automatically creates a timeline of key legal events. By automating these traditionally time-consuming and costly processes, the solution not only boosts productivity and accuracy but also making legal workflows faster, simpler, and more affordable.

Contents

1 Introduction **1**
1.1 Motivation . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 3
1.4 Scope . . . 4

2 Literature Review **6**

3 Project Design **9**
3.1 Existing System . . . 9
3.2 Proposed System . . . 9
3.2.1 Critical Components of System Architecture . . . 9
3.3 System Diagrams . . . 10
3.3.1 UML Diagram . . . 10
3.3.2 Activity Diagram . . . 11
3.3.3 Use Case Diagram . . . 12
3.3.4 Sequence Diagram . . . 13

4 Project Implementation **15**
4.1 Code Snippets . . . 15
4.2 Steps to access the System . . . 20
4.3 Timeline Sem VII . . . 24

5 Testing **26**
5.1 Software Testing . . . 26
5.1.1 Testing Objectives . . . 27
5.1.2 Testing Methodology . . . 27
5.1.3 Test Case Summary . . . 27
5.1.4 Performance Testing Insights . . . 28
5.1.5 Security Testing . . . 28
5.1.6 Overall Outcome . . . 29
5.2 Functional Testing . . . 29

6 Result and Discussions **31**

7 Conclusion **35**

8 Future Scope **36**

List of Figures

1.1 Problem Definition 3

3.1 Proposed System Architecture 10
3.2 UML Classification 10
3.3 Activity Diagram – Chatbot 11
3.4 Activity Diagram – Event Timeline 12
3.5 Use Case Diagram 13
3.6 Sequence Diagram 14

4.1 Configuration & API Setup 15
4.2 Make Gemini API Call() Helper Function 16
4.3 The Chat Endpoint 16
4.4 Extract text from various document types 17
4.5 Find Dates and Events 18
4.6 Upload Files 19
4.7 Get Schedule 19
4.8 Get Schedule 20
4.9 Homepage 21
4.10 Legal Chatbot 21
4.11 Lawyer Event Timeline 22
4.12 Scheduled dates 22
4.13 Risk Analyzer 22
4.14 Risk Analysis Results Dashboard 23
4.15 Legal Document Identification and Draft Generation 23
4.16 Legal Document Drafting Interface 23
4.17 Timeline of the Project Milestone 25

6.1 Comparison of Gemini Flash 3, GPT-4 turbo and llama 3 across evaluation metrics 32
6.2 Extraction Performance 33
6.3 Ordering Accuracy vs Number of Events 33

List of Tables

5.1 Functional Testing Table for LawGrid AI 30

6.1 Model Selection for Document QA System 31
6.2 Evaluation of Chatbot Responses for Document Domain 31
6.3 Result Table for Event timeline Legal Document 32
6.4 Qualitative Comparison of Manual and Automated Document Analysis . . . 34

List of Abbreviations

AI:	Artificial Intelligence
NLP:	Natural Language Processing
UML:	Unified Modeling Language

Chapter 1

Introduction

Legal professionals often deal with multiple contracts, compliance requirements, and deadlines simultaneously. Traditional contract management is time-consuming, requiring manual review of every clause to ensure compliance with laws and company policies. This manual approach increases the likelihood of missing critical risks, such as ambiguous language, unfavorable terms, or overlooked renewal dates. These inefficiencies can lead to significant delays in contract execution, disputes, or even legal liabilities, ultimately costing businesses money and damaging their reputation. Our project addresses this problem by automating contract review and risk identification, minimizing delays and reducing the possibility of human error.

Many legal software solutions in the market focus only on document storage or generic contract templates without adapting to the specific needs of users. They fail to offer real-time guidance or context-aware support, leaving users to interpret complex legal terms on their own. Legal professionals and businesses require solutions that understand the nuances of their contracts and offer advice tailored to their industry or jurisdiction. Our system bridges this gap by using AI and NLP to provide real-time, personalized assistance, ensuring that users receive relevant insights and explanations based on the document they are working with.

The core strength of our solution lies in its ability to simplify the entire contract lifecycle. Users can automatically generate compliant contracts by answering a set of guided questions, reducing the need for repetitive manual drafting. When users upload existing documents, the system uses AI-driven analysis to detect risks, flagging problematic clauses or compliance gaps. More importantly, it goes beyond just pointing out issues — it provides practical, actionable recommendations for resolving them and explains the reasoning in simple language. This helps both legal professionals and non-experts make informed decisions quickly and confidently.

To make legal processes more user-friendly, our platform integrates an interactive chatbot that acts as a virtual legal assistant. This chatbot can answer questions in real time, explain legal terminology in plain English, and guide users step by step through contract creation or review. Additionally, the system automatically builds visual timelines for key legal events — such as deadlines, renewal dates, or obligations — helping users stay organized and proactive. This combination of conversational support and visual clarity streamlines workflows, enhances collaboration, and ensures that important legal tasks are never overlooked.

1.1 Motivation

Traditional legal services and most existing tools rely heavily on manual review processes, which are not only time-consuming but also inconsistent in quality. Legal professionals or users often have to go through lengthy contracts and legal documents line by line, which increases the risk of oversight. Existing tools typically offer generic templates and basic storage solutions, which fail to consider the unique context of each user's situation — such as industry requirements, jurisdiction-specific regulations, or organizational policies. This lack of personalization means users receive one-size-fits-all outputs that may not fully address their legal needs.

When legal tools do not adapt to a user's specific context, the burden of interpretation falls entirely on the user, who may not have legal expertise. This creates room for misunderstanding important clauses, overlooking compliance requirements, or even signing contracts that may carry hidden risks. Legal professionals, meanwhile, spend hours performing repetitive manual reviews instead of focusing on higher-value tasks like strategy and negotiation. As a result, businesses face slower decision-making, increased legal costs, and potential exposure to liabilities — all of which negatively affect productivity and profitability.

In today's fast-paced business environment, there is a growing demand for legal support that is both accurate and context-aware. Users need tools that can analyze their documents in real time and provide insights specific to their situation — whether that's flagging risky clauses, ensuring compliance with the latest regulations, or suggesting better alternatives. Accuracy is critical, as even a small oversight in a contract can lead to significant financial and legal consequences. A system that delivers precise and customized assistance reduces risk, saves time, and gives users confidence in their legal decisions.

Our motivation is to bridge this gap by developing a system that combines the expertise of legal professionals with the efficiency of artificial intelligence. By integrating NLP, machine learning, and conversational AI, the system can deliver personalized, accurate, and real-time legal assistance. Users benefit from an interactive chatbot that not only answers questions but also explains risks in easy-to-understand terms, while automated risk detection and contract generation streamline the entire workflow. This innovation ultimately transforms legal support from a slow, manual process into a fast, accessible, and user-friendly experience.

1.2 Problem Statement

Legal documents such as contracts, compliance records, and case filings are often dense, filled with jargon, and difficult for both legal professionals and non-experts to interpret. This complexity slows down workflows, increases the risk of errors or missed deadlines, and leaves clients or junior staff struggling to understand key details. Existing tools in the legal sector typically focus on storage and management rather than simplifying content or streamlining processes, forcing professionals to rely on manual reviews and drafting.

An effective solution would be an AI-powered system that explains legal terms and clauses in plain language, making documents accessible to all stakeholders. It could automatically track key dates and milestones, flag risks like conflicting clauses or compliance issues, and even generate routine contracts to save time and reduce human error. Integrated chat bot support could provide instant answers, guidance, and workflow assistance, improving transparency and efficiency. By combining AI, automation, and user-friendly design, such a system would modernize legal workflows, reduce barriers created by legal jargon, and make legal processes faster, clearer, and more reliable for professionals and clients alike.

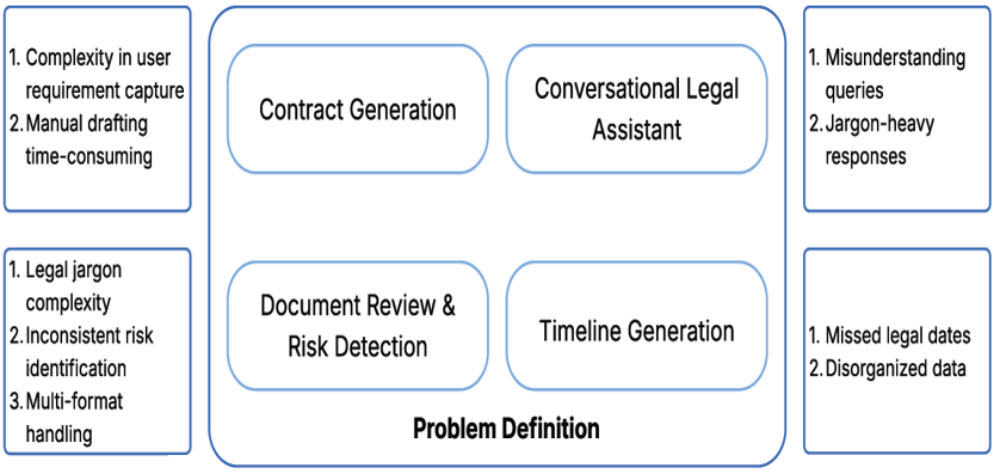

Figure 1.1: Problem Definition

1.3 Objectives

The AI Legal Assistant project aims to transform the way legal processes are managed by Natural Language Processing (NLP). The system is designed to simplify legal document creation, review, and understanding by providing users with automated contract generation, intelligent risk detection, easy-to-follow solutions. It further enhances accessibility through an interactive legal chatbot and organizes key legal events into clear, visual timelines. By leveraging these modern technologies, the solution minimizes human effort, reduces legal risks, improves efficiency, and makes legal processes more transparent and user-friendly for both professionals and non-experts.

- To review user input and generate accurate contract documents efficiently using AI-powered contract generation and Natural Language Processing (NLP): The system simplifies the contract creation process by allowing users to provide structured input through guided fields or questionnaires. Using AI-powered contract generation techniques and predefined legal templates, it transforms this input into legally compliant and error-free documents. This automation ensures consistency, reduces human error, and saves significant time for legal professionals and businesses. By eliminating repetitive manual drafting, the system allows users to focus on higher-value tasks such as strategy and negotiation, while still producing precise, professional-quality contracts.

- To allow users to upload their contracts, identify legal and risk issues, and provide clear explanations along with practical solutions using machine learning and NLP: The platform empowers users to upload existing legal documents for intelligent review. Through Natural Language Processing (NLP) and machine learning-based risk detection models, the system scans the text to identify problematic clauses, compliance gaps, or unfavorable terms. Rather than just flagging issues, it explains them in plain language and suggests actionable solutions or alternative clauses that users can adopt. This approach minimizes the risk of legal disputes, ensures compliance with current regulations, and gives users the confidence to make informed decisions.

- To build a conversational chatbot that functions as a legal assistant, enabling users to communicate their legal needs and comprehend legal information more effectively using conversational AI: The system includes an interactive chatbot powered by conversational AI, which acts as a virtual legal advisor. Users can ask questions in natural language, receive immediate answers, and get guided support through contract creation or document review. The chatbot also breaks down complex legal terminology into easy-to-understand explanations, helping non-experts grasp legal concepts quickly. This real-time interaction ensures that legal support is always accessible, improving user engagement and reducing reliance on human experts for minor clarifications.

- To create an easy-to-understand timeline by automatically finding and arranging important legal events and dates from different documents using AI-driven document parsing: The platform automatically scans uploaded contracts and extracts key dates such as contract start and end dates, renewal deadlines, payment schedules, and compliance obligations. It then organizes these dates into a clear, visual timeline that helps users stay proactive and meet important deadlines. This feature enhances planning, improves compliance tracking, and prevents costly delays or missed obligations. By turning complex legal schedules into a simple, structured format, the system makes legal management more transparent and user-friendly.

1.4 Scope

The scope of the LawGrid project is to develop an intelligent legal assistant system that simplifies the understanding and management of legal documents, enhances workflow efficiency, and provides actionable insights to both legal professionals and non-experts.

- Designed for everyone, individuals, small businesses, and legal professionals: The system is meant to be useful for everyone, regular people who need to understand legal documents, small businesses that want faster and simpler contract handling, and legal professionals who need efficient tools to save time and reduce errors.

- Eliminates legal jargon with clear, plain-language explanations: Converts complicated legal terms and technical language into simple, easy-to-understand wording. This helps

anyone whether they have legal knowledge or not quickly grasp the meaning of documents without confusion or the need for expert help.

- Provides smart assistance through chat bot, document review, and summaries: Offers intelligent support features such as a chat bot to instantly answer common legal questions, automated document review to spot potential issues or risks, and concise summaries to highlight key points. These tools save time, reduce errors, and make handling legal documents much more convenient.

- Makes legal processes faster, easier, and more transparent: Speeds up tasks such as drafting, reviewing and managing legal documents while simplifying every step of the workflow. By improving visibility into timelines, changes, and risks, it ensures that all parties clearly understand the process, creating a smoother and more open legal experience for everyone involved.

Chapter 2

Literature Review

This literature review explores the body of research on LawGrid:AI Legal Assistant For Antology-Driven Legal Reasoning And Support,examining the key studies, theories, and findings that have shaped the field. The review provides a critical analysis of existing work, highlighting significant contributions as well as areas where knowledge remains limited or inconclusive. By analyzing the approaches and conclusions drawn by various scholars, this review aims to clarify current understandings and identify potential directions for future research.

[1] **Accurate AI Assistance in Contract Law Using Retrieval-Augmented Generation to Advance Legal Technology (2025).** This study by Youssra Amazou, Faouzi Tavalati, Houssam Mensouri, Abdellah Azmani, and Monir Azmani explores the application of Retrieval-Augmented Generation (RAG) systems in legal contract analysis. The methodology involves converting legal documents into vector embeddings stored in a database. When a user poses a query, the system retrieves relevant document chunks and leverages a Large Language Model (LLM) to generate an informed response. The research highlights a critical limitation: the system's accuracy and utility are directly dependent on the completeness and quality of the legal databases. If the necessary information is missing or not explicitly documented in the legal texts, the system fails to provide a relevant answer and returns "Not applicable." This identifies a significant research gap, emphasizing the need for richer, more comprehensive datasets to enhance AI-assisted legal decision-making.

[2] **AI-Powered Legal Documentation Assistant (2024).** P. Vimala Imogen, J. Sreenidhi, and V. Nivedha present a system aimed at simplifying the legal documentation process through AI integration. The methodology employs a combination of HTML, CSS, JavaScript, and PHP for the user interface and backend, supported by a MySQL database for data and template storage. The core AI capability is implemented through a Collect.chat-based chatbot that automates document generation using predefined templates. Despite its efficiency in template-driven document creation, the research identifies a major limitation: the system still requires manual editing of contracts. This step introduces the risk of human errors such as typos or inconsistencies, which can delay the finalization of legal documents. The study highlights the need for more advanced automation techniques that minimize human intervention and ensure accuracy in contract preparation.

[3] **Bettercall: AI-Based Legal Assistant (2024).** Manirao Dhore, Aryan Vimal,

Avnish Agrawal, Raghav Bajaj, and Rohan Barde developed Bettercall, an AI-driven legal assistant designed to streamline the retrieval of legal information. The system functions in two stages: first, it converts collected legal documents into vector embeddings, which are stored in a database. In the second phase, user queries are transformed into embeddings and matched against the database using Cosine Similarity to identify the most relevant legal documents. While the system demonstrates efficiency in information retrieval, its performance faces significant challenges when dealing with India's multilingual legal data. Since a large proportion of legal texts exist in regional languages, the AI may not always interpret or process them accurately. Additionally, large language models (LLMs) have a known tendency to provide inaccurate or incomplete results, which presents another limitation in ensuring reliability.

[4] **AI-Powered Legal Documentation Assistant (2024).** Kuldeep Vayadande, Aditya Bhovar, Aditi Bhat, Zulfikar Charoliya, Pranav Bachhav, and Aavush Chavan introduced an AI-powered documentation assistant with advanced features that go beyond template generation. The system employs the ChatGPT-3.5 model, trained with diverse legal documents and strengthened through OpenAI embeddings. It integrates multiple technologies including PyPDF, Amazon Textract, and LangChain to process legal documents, while Natural Language Processing (NLP) techniques such as sentiment analysis and Named Entity Recognition (NER) are used to identify irregularities. Despite its innovative approach, the research highlights several limitations. Existing legal tools often lack personalization, real-time assistance, and the capability to handle complex contract management. A major research gap lies in creating integrated systems capable of not only detecting compliance risks but also providing clear, explainable reasoning for their outputs—an essential requirement in legal practice.x

[5] **LEGAL AI: A Tool to Help Lawyers Optimize their Performance (2024).** Allan Almeida, Shubh Shah, Neer Bhavesh Khandor, and Neha Agarwal developed LEGAL AI, an AI-powered platform aimed at assisting lawyers by integrating multiple advanced technologies into a single system. The solution combines Named Entity Recognition (NER) models for document redaction, translation models for multilingual accessibility, and large language models (LLMs) for legal question answering. These modules are interconnected through semantic indexing and APIs, creating an end-to-end AI-based legal assistant. While the system shows significant promise in optimizing legal workflows, a major limitation lies in the lack of a standardized framework for evaluating redaction and translation quality. Furthermore, existing legal AI systems rarely combine privacy, translation, and legal QA into one cohesive solution, highlighting a key research gap.

[6] **AI-Driven Document Automation and Compliance in Contract Lifecycle Management (2024).** Sweta Privadarshni presented a system focused on automating legal document management and compliance checking throughout the contract lifecycle. The approach leverages NLP techniques such as clause extraction, transformer-based models, and machine learning classifiers trained on datasets like CUAD to ensure accuracy in compliance verification. Additionally, the system integrates adaptive AI for drafting, reviewing, and monitoring legal documents. Despite its advancements, the study identifies challenges in ensuring data privacy, explainability, and meeting evolving regulatory standards. Another limitation is that current models lack adaptability to dynamic, multi-jurisdictional environments, which restricts their practical deployment in diverse legal contexts.

[7] Big Data and AI-Driven Evidence Analysis: A Global Perspective on Citation Trends, Accessibility, and Future Research in Legal Applications (2023). Divya Mehta, Tanvi Malhotra, Arpan Kumar, and P.K. Jain presented an AI-based approach for improving evidence analysis and citation trends in legal studies. Their system employs Collect.Chat, an NLP-powered chatbot designed for document analysis and template recommendations. It also integrates a Code Mirror editor to enable AI-assisted template customization with enhanced legal accuracy. While the system demonstrates potential in automating evidence handling, it faces notable limitations. Specifically, the chatbot has limited contextual understanding compared to large language models (LLMs) and relies on predefined templates instead of generating complete legal documents. This restricts its flexibility and makes it slower in addressing diverse user queries.

[8] An Approach to Get Legal Assistance Using Artificial Intelligence (2023). Nishant Jain and Gaurav Goel proposed a Virtual Legal Assistant built on a Service-Oriented Architecture comprising four main components: text analytics for preprocessing case data, similarity and ranking mechanisms for case retrieval, a knowledge base for storing rules and models, and a chatbot interface for speech/text interactions. The system effectively processes legal data and provides ranked outputs, supporting basic legal query handling. However, its reliance on semantic networks and embedding models like Word2Vec/Doc2Vec limits its scope to predefined legal case retrieval. As a result, the system cannot fully address diverse or complex legal queries. This gap underscores the need for more advanced LLM-powered solutions capable of broader, context-aware legal assistance.

[9] Artificial Intelligence and Law: An Effective and Efficient Instrument (2020). Dr. Hemlata Sharma and Aakanksha explored the role of Artificial Intelligence in reshaping the legal sector through a descriptive and analytical approach. Their study combines literature reviews, thematic analyses, and case studies to investigate AI applications across different branches of law. The paper also compares India's AI legal framework with global practices and concludes with policy recommendations. While the research highlights the conceptual importance of AI in legal transformation, it is limited largely to theory and policy discussion. The absence of practical AI tools for contract review, risk detection, or litigation support creates a major gap. Furthermore, the study does not propose interactive chatbot systems or automated legal timeline generation, restricting its utility for real-world applications.

[10] Use of Artificial Intelligence to Analyse Risk in Legal Documents for Better Decision Support (2018). Dipankar Chakrabarti, Neelam Pattodia, Udayan Bhattacharya, Indranil Mitra, Satyaki Roy, Jayanta Mandi, Nandini Roy, and Prasun Nandy developed an AI-driven framework aimed at analyzing legal risks within documents. Their system employs Retrieval-Augmented Generation (RAG), where legal texts are transformed into vector embeddings and stored in a database. When a user submits a query, the model identifies relevant document chunks through similarity search and generates responses using a large language model (LLM). Although the system improves decision support by connecting semantic search with generative AI, its accuracy is highly dependent on the quality and completeness of the underlying database. If critical information is missing or incomplete, the model may fail to provide useful or reliable answers.

Chapter 3

Project Design

3.1 Existing System

The existing systems for legal documentation and contract management primarily rely on manual drafting, traditional text editors, or limited software tools. While these approaches provide basic functionality, they suffer from several drawbacks. For instance, manual contract drafting is time-consuming and prone to human errors such as inconsistencies or missing clauses. Similarly, existing legal assistance tools often lack automation in risk assessment, making it difficult for users to detect liabilities or compliance issues.

Moreover, most current solutions do not offer integrated features such as automated contract review, evidence-based legal QA, or timeline generation for case tracking. This lack of intelligent automation makes it challenging for lawyers, businesses, and individuals to achieve efficiency, accuracy, and faster decision-making in legal processes.

3.2 Proposed System

3.2.1 Critical Components of System Architecture

The proposed system architecture for Lawgrid is designed as an AI-driven legal assistant accessible through a web application. It integrates multiple modules—Contract Drafting, Contract Review, Legal QA, and Event Timeline—to simplify and automate complex legal processes. In the drafting module, the system collects user inputs through forms and NLP-based intent detection, selects suitable templates, and generates editable drafts using LLMs with compliance checks.

The Legal QA module enables users to ask law-related questions in natural language and receive precise, evidence-based answers using NLP and LLMs. Additionally, the Event Timeline feature extracts important dates and events from documents and arranges them chronologically for better case tracking. Together, these components create a robust legal assistance platform that reduces manual effort, minimizes errors, and provides accessible, AI-powered solutions for legal documentation, review, and case management.

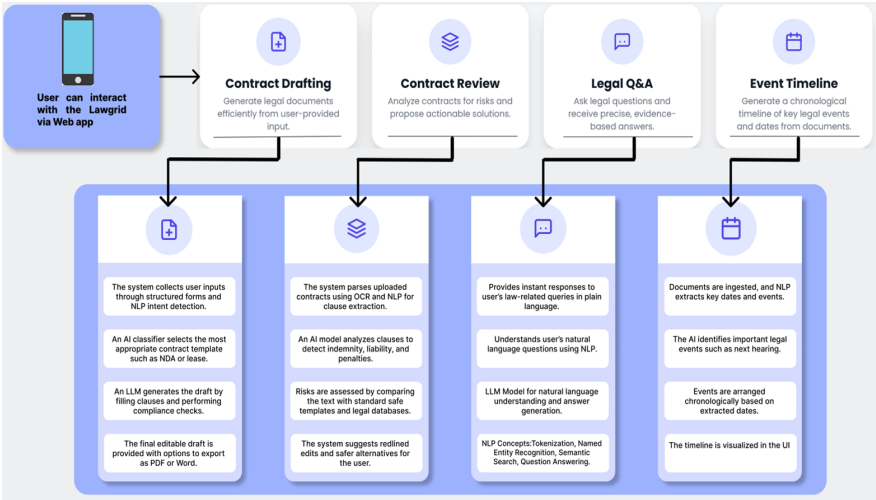

Figure 3.1: Proposed System Architecture

3.3 System Diagrams

3.3.1 UML Diagram

Unified Modeling Language (UML) is a general-purpose modeling language. The main aim of UML is to define a standard way to visualize the way a system has been designed.UML is linked with object-oriented design and analysis. UML makes use of elements and forms associations between them to form diagrams.

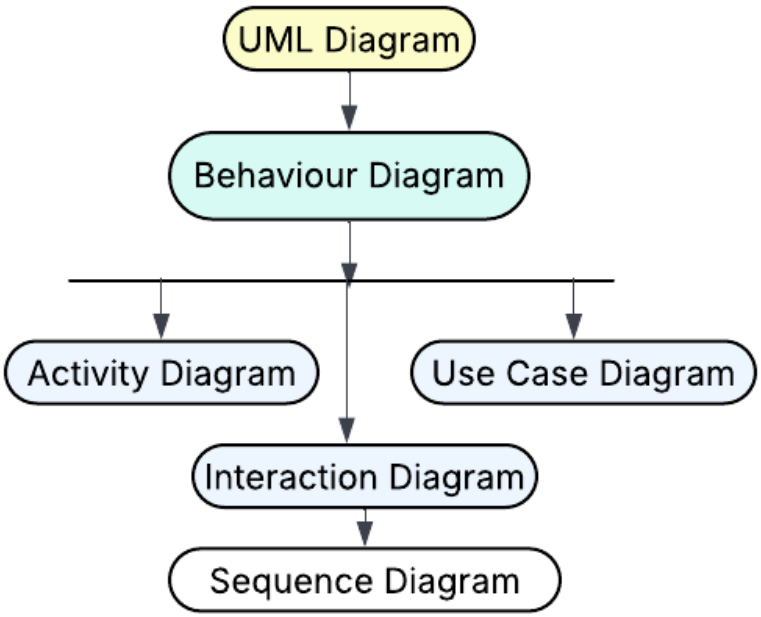

Figure 3.2: UML Classification

3.3.2 Activity Diagram

We use Activity Diagrams to illustrate the flow of control in a system. We can also use an activity diagram to refer to the steps involved in the execution of a use case.

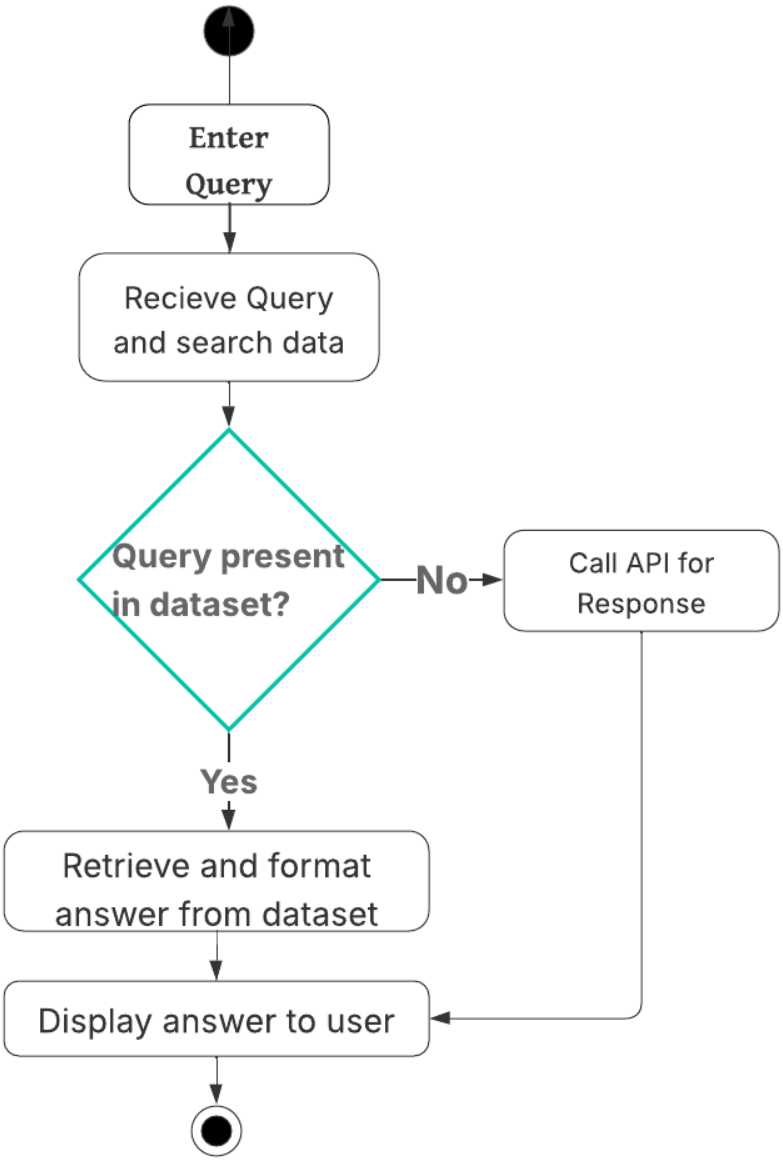

Figure 3.3: Activity Diagram – Chatbot

The provided activity diagram illustrates the process of how a chatbot handles a user query. The process begins when a user enters a query. The system then receives the query and searches its existing dataset for a relevant answer. A decision point is reached: if the query is present in the dataset, the system retrieves and formats the pre-existing answer. If the query is not found, the system instead calls an API to generate a response. Regardless of the path taken, the final step is to display the answer to the user, which concludes the

process.

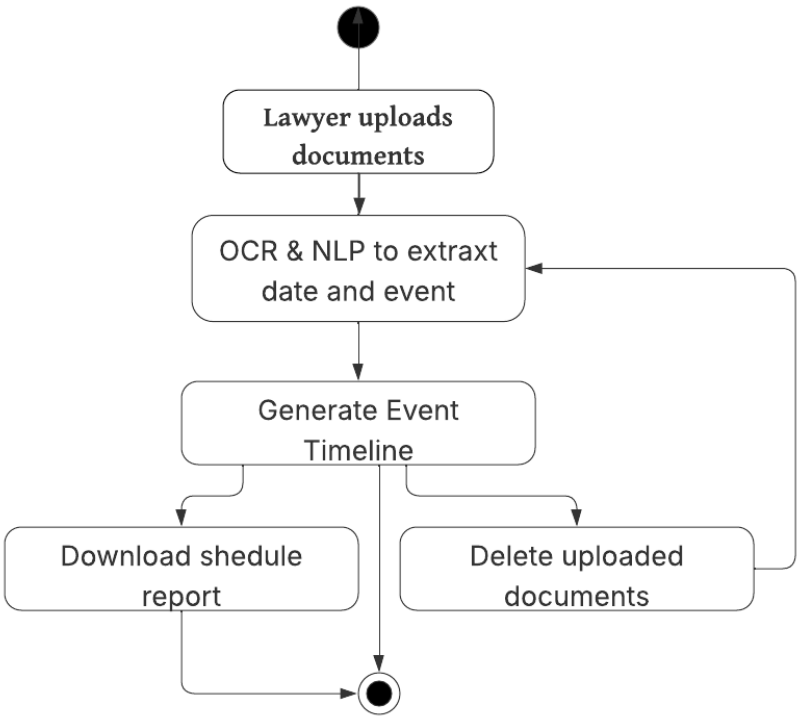

Figure 3.4: Activity Diagram – Event Timeline

The figure 3.3 shows the process of creating an event timeline from legal documents.The process starts when a lawyer uploads documents. The system then uses OCR (Optical Character Recognition) and NLP (Natural Language Processing) to extract important dates and events from the documents. After this processing, the system generates an event timeline. From here, the lawyer has three options: they can download a schedule report, delete the uploaded documents, or go back to the OCR and NLP step, perhaps to refine the process or add more documents. The process concludes once the user has taken one of these actions.

3.3.3 Use Case Diagram

The Use Case Analysis chapter provides a detailed overview of the user interactions and functional requirements of the Lawgrid System. This chapter highlights the different use cases, the primary actors involved, and their relationships, illustrating how users and lawyers engage with the system to efficiently handle legal documentation, review, and case management. The main actors in this system are the User and the Lawyer, both of whom interact with the Lawgrid platform through a web application that supports multiple AI-powered legal functionalities.

As demonstrated in Figure 3.3, the key use cases include Contract Drafting, where users and lawyers can generate legal documents using AI-driven templates. This involves structured input collection, automatic clause generation, and compliance checks, after which the contract can be exported in PDF or Word format. The Contract Review use case allows the system to analyze uploaded legal documents, detect liabilities, risks, and penalties, and suggest safer alternatives. This process includes a dedicated Risk Compliance Check, which detects inconsistencies and ensures that contracts align with standard legal frameworks.

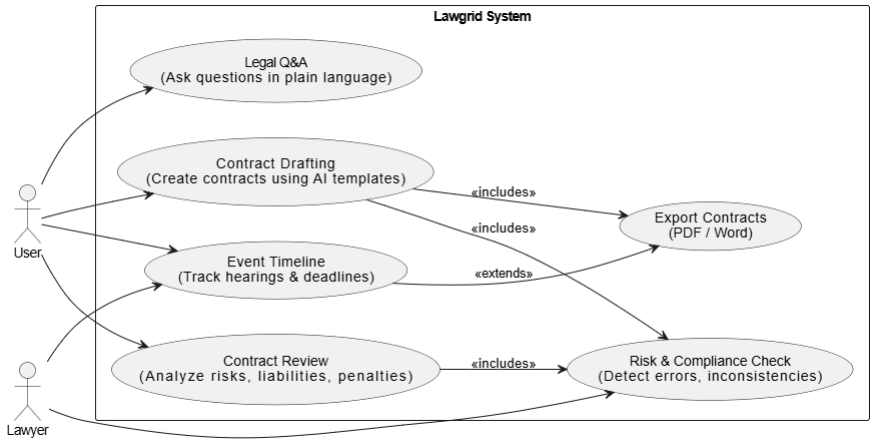

Figure 3.5: Use Case Diagram

The Legal QA use case enables both users and lawyers to ask law-related queries in plain language, receiving precise, evidence-based answers through Natural Language Processing (NLP) and Large Language Models (LLMs). Another important functionality is the Event Timeline, which automatically extracts dates and deadlines from documents, arranges them chronologically, and generates an interactive timeline of hearings and legal events.

Each of these use cases is interconnected, forming a comprehensive AI-powered legal assistant that not only assists with drafting and reviewing contracts but also supports legal research and case tracking. The integration of advanced technologies such as NLP, OCR, embeddings, and LLMs ensures that the system delivers accurate, reliable, and accessible legal solutions. In conclusion, the Use Case Analysis chapter underscores how Lawgrid simplifies legal workflows by reducing manual effort, minimizing errors, and providing an intelligent, evidence-based legal support system for both users and lawyers.

3.3.4 Sequence Diagram

A sequence diagram simply depicts interaction between objects in a sequential order i.e. the order in which these interactions take place.

Figure 3.6 illustrates the automated flow of a user query through a chat bot system. The process begins when a user sends a query to the chatbot's system. Upon receiving the query, the system first checks its dataset for a direct match. Based on this check, one of two alternative paths is taken: if an answer is found, the system immediately retrieves and

sends the response back to the user; however, if no answer is found, the system automatically makes a call to an external API, which then provides the answer that is ultimately returned to the user.

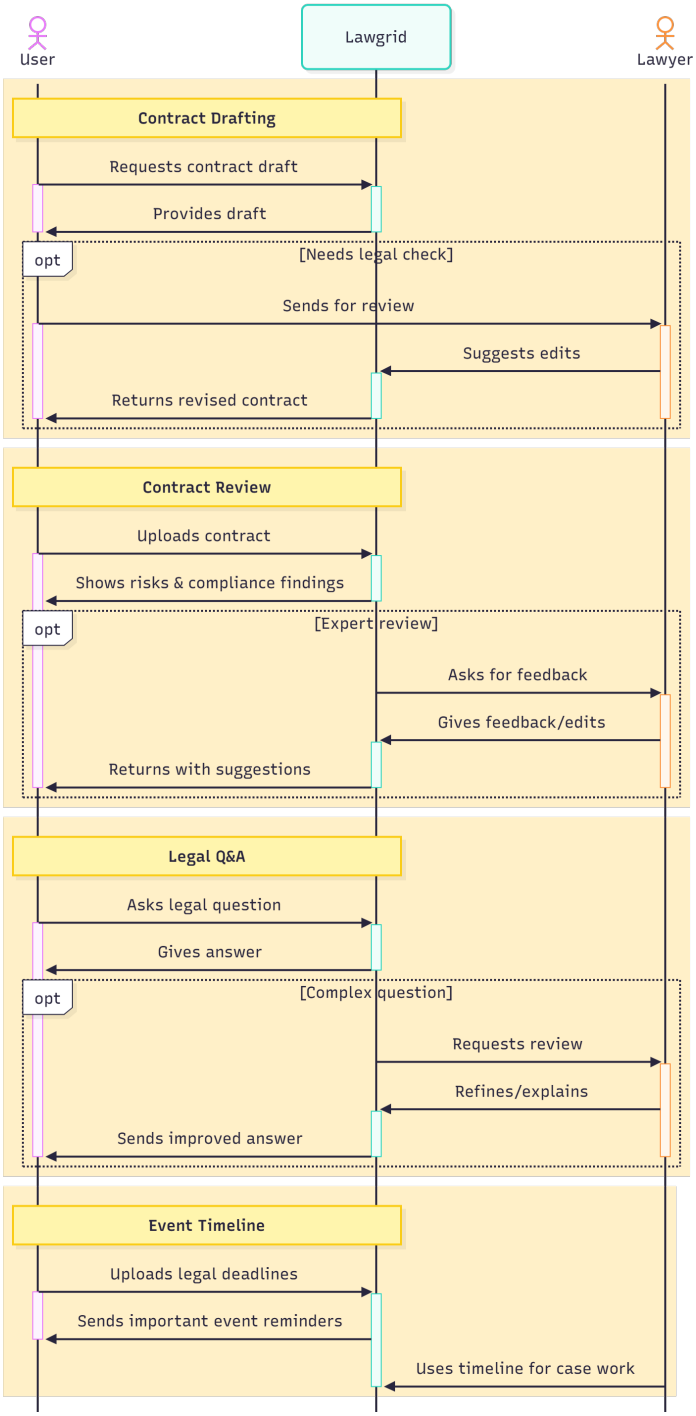

Figure 3.6: Sequence Diagram

Chapter 4

Project Implementation

4.1 Code Snippets

To better understand the implementation of the Lawgrid system, this section includes important code snippets that highlight key components of the project. These examples demonstrate how various modules—such as contract drafting using AI templates, clause risk detection and explanation with NLP, legal case outcome prediction using machine learning, smart event timeline generation, cross-law mapping, and case law visualization—are developed and integrated to work together. Each snippet is carefully selected to show the core logic and functionality behind the intelligent, interactive, and user-friendly legal assistance offered by the system.

This code in Figure 4.1 illustrates the configuration setup that enables the integration of advanced AI models within the Lawgrid system. This initial step ensures that the platform is connected to the external language processing service through secure authentication. Without this setup, the system would not be able to access the intelligence required for functions such as contract drafting, clause analysis, and predictive legal insights.

```
# --- Configuration ---
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
if not GEMINI_API_KEY:
    raise ValueError("GEMINI_API_KEY not found in .env file. Please set it.")

GEMINI_TEXT_MODEL = "gemini-2.5-flash-preview-05-20"
GEMINI_TTS_MODEL = "gemini-2.5-flash-preview-tts"
GEMINI_API_BASE_URL = "https://generativelanguage.googleapis.com/v1beta/models"
```

Figure 4.1: Configuration & API Setup

Alongside text processing, another model is designated for converting system outputs into spoken language. This feature strengthens accessibility within Lawgrid, allowing users—lawyers, clients, or law students—to interact with the system through a voice-enabled interface. It enhances usability, especially for quick clause explanations, case law insights, or timeline updates delivered in real-time speech.

Finally, the configuration specifies the communication pathway to the external AI service provider. This ensures that every request and response follows a stable, predefined route, enabling smooth integration of multiple modules in Lawgrid. Collectively, the parameters shown in Figure 4.1 form the foundation upon which advanced legal functions—ranging from risk detection to event tracking—operate reliably and securely.

This code mentioned in Figure 4.2 presents the method responsible for handling secure and reliable communication between the Lawgrid system and the external AI service. Since Lawgrid heavily relies on advanced AI models for drafting, clause risk detection, and case analysis, this function ensures that every request sent to the model and every response received is processed correctly. It acts as the backbone for maintaining smooth interaction with the AI engine.

```python
def make_gemini_api_call(url, payload, max_retries=5, base_delay=2):
    retries = 0
    while retries < max_retries:
        try:
            headers = {'Content-Type': 'application/json'}
            response = requests.post(url, headers=headers, data=json.dumps(payload))
            response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)
            return response.json()
        except requests.exceptions.HTTPError as e:
            if response.status_code == 429: # Too Many Requests
                delay = base_delay * (2 ** retries)
                print(f"Rate limit hit. Retrying in {delay} seconds... (Attempt {retries + 1}/{max_retries})")
                import time
                time.sleep(delay)
                retries += 1
            else:
                print(f"HTTP Error: {e.response.status_code} - {e.response.text}")
                raise
        except requests.exceptions.RequestException as e:
            print(f"Request Error: {e}")
            delay = base_delay * (2 ** retries)
            print(f"Request failed. Retrying in {delay} seconds... (Attempt {retries + 1}/{max_retries})")
            import time
            time.sleep(delay)
            retries += 1
    raise Exception("Max retries exceeded for API call.")
```

Figure 4.2: Make Gemini API Call() Helper Function

The method incorporates error handling to deal with unexpected failures. For instance, if the AI service returns an error due to incorrect requests or server issues, the system detects it immediately and prevents unreliable outputs. This is critical in a legal setting where incomplete or incorrect responses could lead to serious misinterpretations. By raising explicit error messages, the system ensures transparency in its operation.

```python
def chat():
    """Handles chat messages and calls the Gemini text generation API."""
    user_message = request.json.get('message', '').lower() # Convert to lowercase for case-insensitive check
    if not user_message:
        return jsonify({"error": "No message provided"}), 400
```

Figure 4.3: The Chat Endpoint

This code shows in Figure 4.3 illustrates the function that manages user interactions within the Lawgrid system. This method serves as the entry point for processing chat messages, which are the primary medium through which users communicate with the platform.

By capturing input from the user, it ensures that queries—such as requests for contract drafting, risk assessment, or legal case explanations—are recognized and prepared for further processing.

The function begins by retrieving the message submitted in the chat interface. This incoming text is then converted into a consistent format by lowering all characters. Such normalization is important in natural language processing, as it allows the system to interpret user intent correctly without being affected by variations in capitalization. For example, whether a user types "Review Contract" or "review contract", the system treats both requests equally.

Once a valid message is captured, the function is designed to route this request to the AI text generation model, which then provides context-aware legal responses. This workflow highlights the conversational layer of Lawgrid, making it an interactive legal assistant rather than just a static document analysis tool. By integrating real-time message handling with AI-driven responses, Figure 4.3 represents the foundation of Lawgrid's intelligent legal dialogue system.

This code in Figure 4.4 demonstrates the helper function that enables Lawgrid to extract text from multiple types of legal documents. Since legal work often involves reviewing diverse file formats—such as agreements in Word, scanned judgments in PDF, or simple notes in plain text—this function provides the system with the capability to handle and process them seamlessly.

```
# Helper function to extract text from various document types
def extract_text(file_path, file_extension):
    """
    Extracts text from a given file path based on its extension.
    Supports .docx, .pdf, image formats (.jpg, .jpeg, .png), and .txt.
    """
    try:
        if file_extension in ['.docx']:
            # Uses docx2txt for .docx files
            return docx2txt.process(file_path)
        elif file_extension in ['.pdf']:
            # Uses pdfplumber for .pdf files
            text = ""
            with pdfplumber.open(file_path) as pdf:
                for page in pdf.pages:
                    text += page.extract_text() or ""
            return text
```

Figure 4.4: Extract text from various document types

In the case of PDF files, the function leverages a text extraction tool capable of reading page-by-page content. This feature is critical because many legal documents, such as court judgments or government notifications, are distributed in PDF form. By iterating through each page, the system gathers all available text, making it ready for further tasks like clause risk detection or case law mapping.

The helper function also accounts for plain text and image-based inputs. For .txt files, the extraction process is straightforward, while image formats rely on text recognition methods that can convert scanned contracts or case exhibits into machine-readable form.

```python
# Helper function to find dates and events in the text
def find_dates_and_events(text, source_doc):
    """
    Finds dates and associated event types in the extracted text.
    Updates the database with new events.
    """
    with sqlite3.connect(DATABASE) as conn:
        cursor = conn.cursor()

        # Remove existing entries for this source document
        cursor.execute('DELETE FROM events WHERE source_doc = ?', (source_doc,))
        conn.commit()

        # Define a list of date formats to check for
        date_formats = [
            (r'(?:January|February|March|April|May|June|July|August|September|October|November|December)\s+\d{1,
            (r'\b\d{2}-\d{2}-\d{4}\b', '%m-%d-%Y'),
            (r'\b\d{2}/\d{2}/\d{4}\b', '%d/%m/%Y'),
            (r'\b\d{4}-\d{2}-\d{2}\b', '%Y-%m-%d'),
        ]
```

Figure 4.5: Find Dates and Events

This code in Figure 4.5 presents the helper function that identifies important dates and associated legal events from extracted text. Since legal proceedings often revolve around deadlines, filing dates, and hearing schedules, this function ensures that Lawgrid can automatically recognize and capture such information from uploaded documents. By systematically detecting and organizing these details, the system strengthens its ability to generate accurate legal timelines.

The function begins by connecting to the database where event information is stored. To maintain consistency, it removes any older entries linked to the same source document before inserting new data. This prevents duplication and ensures that the timeline reflects the most up-to-date information whenever a document is re-processed. Such a process is essential for dynamic legal workflows, where revised petitions or amended contracts may alter key deadlines.

Through this intelligent detection and storage process, Figure 4.5 equips Lawgrid with the capability to transform raw legal text into actionable event schedules. This enhances case tracking, deadline monitoring, and overall workflow management, ensuring that users never miss critical dates in their legal proceedings.

This code shows in Figure 4.6 illustrates the function responsible for handling the upload of legal documents into the Lawgrid system. This function forms a critical entry point in the workflow, as it allows users—whether lawyers, clients, or researchers—to provide contracts, petitions, or case records for automated processing.

The process begins by validating the request to confirm that a document has actually been provided. If no file is detected, or if a file is submitted without a valid name, the function immediately returns an error message. Such validation steps are essential for avoiding

```python
@app.route('/upload', methods=['POST'])
def upload_file():
    if 'document' not in request.files:
        return jsonify({'error': 'No document part in the request'}), 400
    file = request.files['document']
    if file.filename == '':
        return jsonify({'error': 'No selected file'}), 400
    if file:
        file_path = os.path.join(app.config['UPLOAD_FOLDER'], file.filename)
        file.save(file_path)

        file_ext = os.path.splitext(file.filename)[1].lower()
        extracted_text = extract_text(file_path, file_ext)
        find_dates_and_events(extracted_text, file.filename)
```

Figure 4.6: Upload Files

system errors and ensuring that Lawgrid only processes legitimate and complete inputs.

This functionality bridges the gap between raw documentation and structured data, enabling downstream modules such as legal timeline generation, clause risk analysis, and predictive case insights.

```python
@app.route('/schedule', methods=['GET'])
def get_schedule():
    with sqlite3.connect(DATABASE) as conn:
        cursor = conn.cursor()
        cursor.execute('SELECT date, event, source_doc FROM events ORDER BY date')
        schedule_items = [{'date': row[0], 'event': row[1], 'source_doc': row[2]} for row in cursor.fetchall()]
    return jsonify({'schedule': schedule_items}), 200
```

Figure 4.7: Get Schedule

This code in Figure 4.7 demonstrates the function responsible for retrieving and displaying the legal schedule stored within the Lawgrid system. This function serves as the interface for users to access organized timelines of events, deadlines, and hearings extracted from uploaded documents. By providing a structured view of key dates, it supports effective case management and ensures that no critical legal deadlines are overlooked.

The method begins by connecting to the system's database, where all events detected from documents are stored. It executes a query to fetch the relevant details—such as the date, type of event, and the source document—from the database. Importantly, the results are ordered chronologically, allowing users to quickly visualize upcoming deadlines and past activities in sequence.

After retrieving the data, the function formats it into a structured list of schedule items. Each item contains essential attributes, including the date, the event description, and the originating document. This structure ensures that the timeline is both human-readable and machine-friendly, facilitating potential integration with other modules such as notifications or calendar exports.

Finally, the schedule is returned as a JSON response, making it easily consumable by web interfaces or other components of the Lawgrid system. Through this functionality, Figure

4.7 completes the loop of event management—from document ingestion and date extraction to structured display—empowering users to track legal timelines efficiently and reliably.

```python
analysis_results = []

# 3️⃣ Analyze EACH clause
for clause in clauses:
    clause_type = detect_clause_type(
        clause["heading"],
        clause["content"]
    )

    if clause_type == "general":
        continue

    # 4️⃣ Base risk report (KB-driven)
    report = generate_final_risk_report(clause_type)

    # 5️⃣ AI explanation (controlled LLM layer)
    explanation = refine_explanation(
        clause_text=clause["content"],
        clause_type=clause_type,
        severity=report["severity"],
        base_risks=report["key_risks"],
        base_recommendations=report["recommended_improvements"]
    )

    report["explanation"] = explanation

    analysis_results.append({
        "heading": clause["heading"],
        "content": clause["content"],  # 🔥 important
        "type": clause_type,
        "report": report
    })
```

Figure 4.8: Get Schedule

This figure demonstrates the backend logic used for clause-level risk analysis in the Law-Grid system. The code iterates through each extracted clause from the uploaded legal contract and determines its clause type by analyzing both the clause heading and content. General or non-critical clauses are skipped to optimize processing efficiency.

Once a relevant clause is identified, the system generates a base risk report using a knowledge-driven approach. This report includes the severity level, key legal risks, and recommended improvements associated with the detected clause type. This step ensures that risk assessment is grounded in predefined legal rules rather than free-form text generation.

After generating the base risk report, the system refines the explanation using a controlled AI layer. The explanation is tailored to the specific clause text, severity level, and identified risks, resulting in a clear and context-aware explanation for the user. Finally, all analyzed clause details—including heading, content, clause type, and risk report—are stored in a structured format, enabling seamless integration with the user interface and PDF highlighting modules.

4.2 Steps to access the System

This chapter presents the detailed implementation of the Law Grid system, focusing on the design and development of its core functional modules. It explains how different components

such as legal document identification, automated document drafting, contract risk analysis, and document export are implemented using Natural Language Processing (NLP) and Large Language Models (LLMs). The chapter also highlights key implementation screenshots and technically critical interfaces that demonstrate the practical working, system flow, and real-time outputs of the proposed solution.

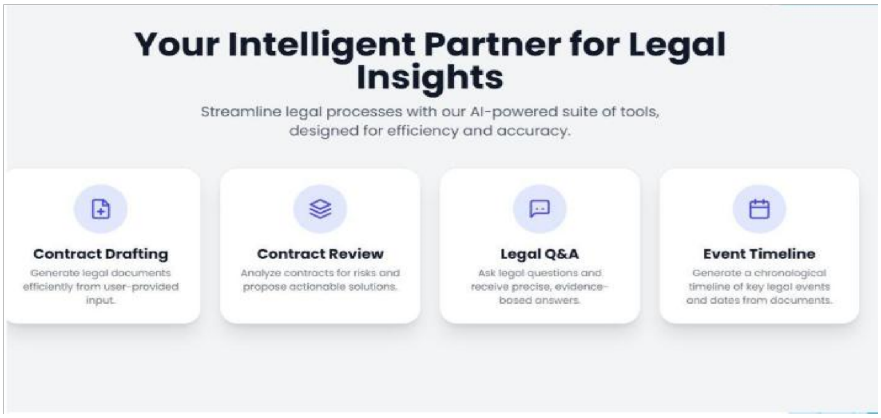

Figure 4.9: Homepage

The Figure 4.8 shows the main landing page of the Law Grid system. It provides centralized access to all core modules including Contract Drafting, Contract Review, Legal QA, and Event Timeline, enabling intuitive navigation for both legal professionals and non-legal users.

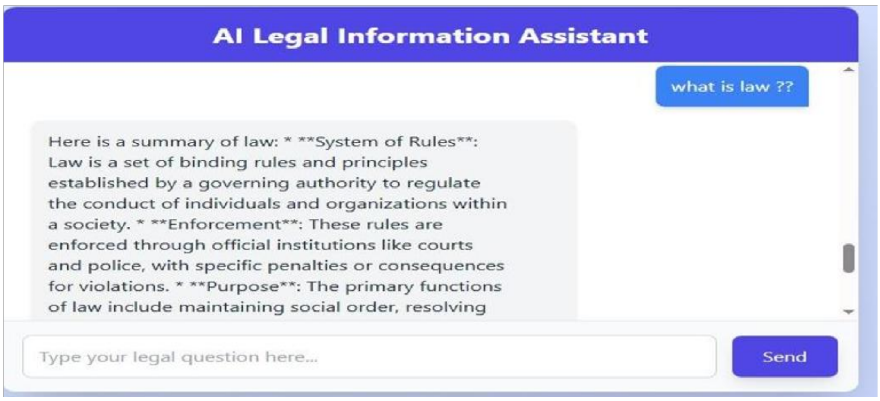

Figure 4.10: Legal Chatbot

This Figure 4.9 illustrates the AI-powered legal chatbot that allows users to ask law-related queries in natural language. The chatbot retrieves domain-specific legal information and responds with clear explanations while restricting answers strictly to legal contexts.

This Fig.4.10 demonstrates the lawyer event scheduler module where legal documents are uploaded. The system applies NLP-based text extraction and named entity recognition to identify important legal events and relevant dates from the uploaded documents.

The Figure 4.11 displays the automatically generated chronological schedule of legal events extracted from documents. Events are sorted in descending order by date, helping

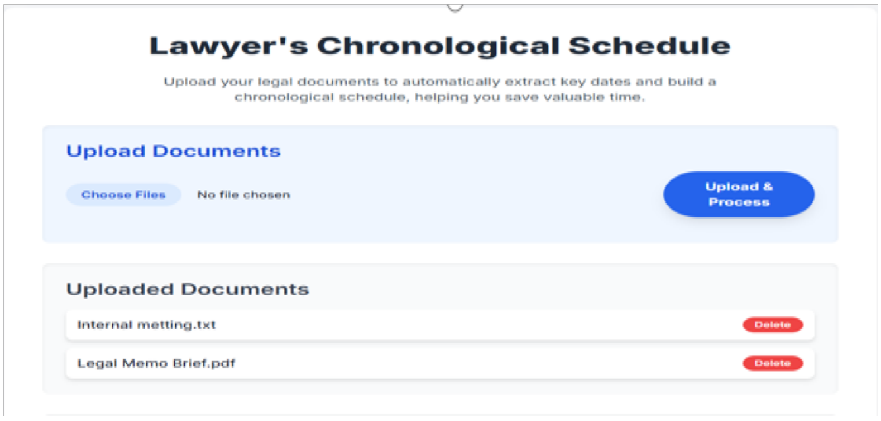

Figure 4.11: Lawyer Event Timeline

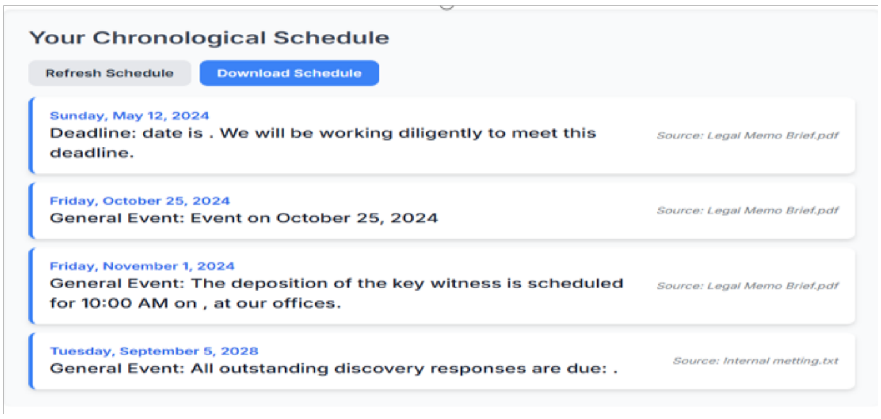

Figure 4.12: Scheduled dates

lawyers efficiently track deadlines, hearings, and important legal milestones.

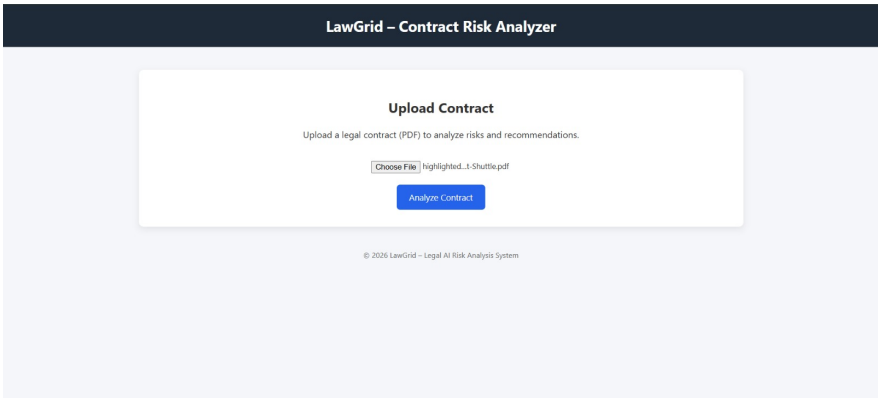

Figure 4.13: Risk Analyzer

This interface (i.e Fig.4.12) enables users to upload legal contracts in PDF format for automated risk assessment. Once uploaded, the system processes the document using NLP techniques and initiates clause-level risk detection and analysis.

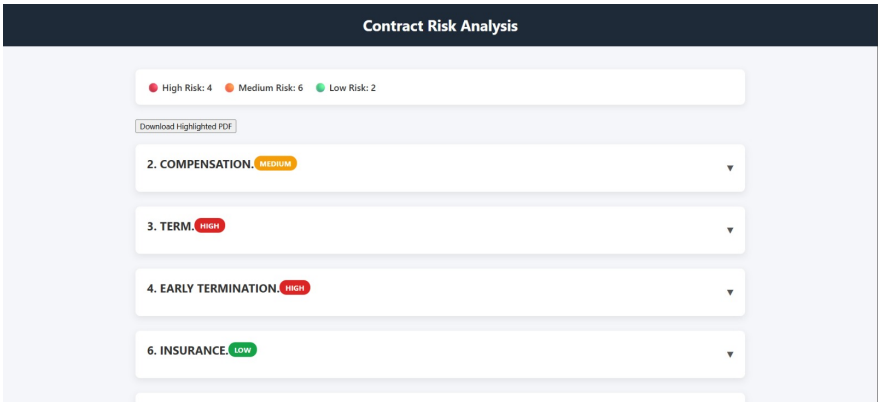

Figure 4.14: Risk Analysis Results Dashboard

This Figure 4.13 presents the analyzed contract clauses categorized into high, medium, and low risk levels. The system highlights risky clauses, assigns severity labels, and provides a structured overview to help users quickly identify critical legal risks.

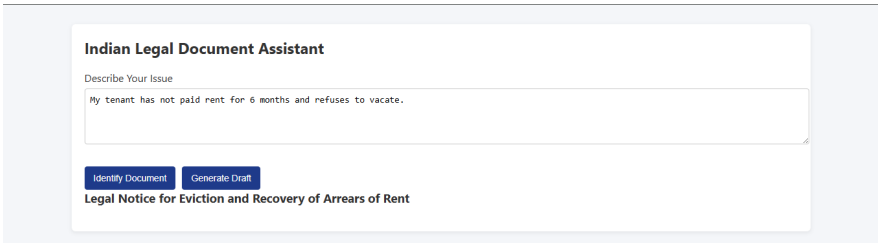

Figure 4.15: Legal Document Identification and Draft Generation

This interface(Figure 4.14) allows users to describe their legal issue in natural language. Based on the problem statement, the system automatically identifies the appropriate legal document type and generates a draft relevant to the user's legal scenario.

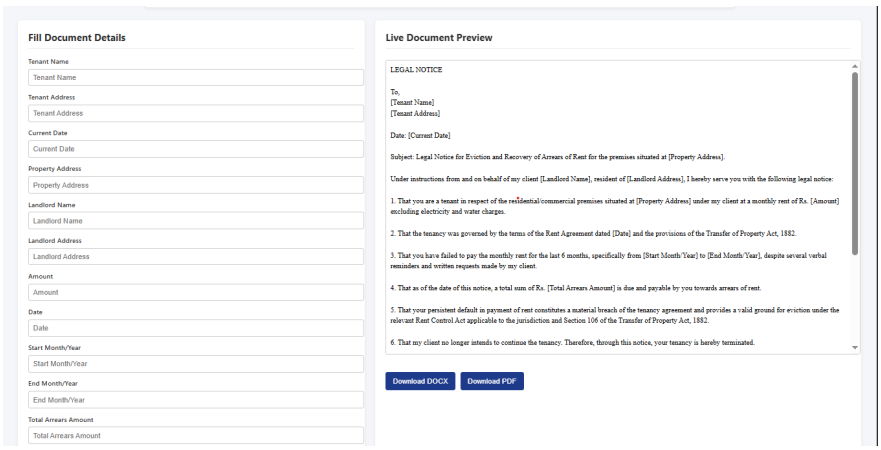

Figure 4.16: Legal Document Drafting Interface

This screen shows the legal document drafting module where users input structured de-

tails such as tenant, landlord, property, and arrears information. The system dynamically generates a legally formatted notice in real time with a live preview and allows exporting the document in DOCX or PDF format.

4.3 Timeline Sem VII

In the context of the Smart Legal AI System (Lawgrid) project, meticulous scheduling played a vital role in ensuring smooth development, timely execution, and seamless collaboration among all team members. The project timeline was carefully designed to define milestones, allocate responsibilities, and maintain an organized workflow across four major phases Project Conception and Initiation, System Design, Implementation, and Testing Validation. Each task was planned based on its complexity, interdependencies, and significance within the overall system. All deadlines were mutually decided after team discussions, considering academic schedules, technical feasibility, and module dependencies. The team adopted a disciplined, collaborative approach to ensure timely completion of each milestone. Weekly review meetings and continuous progress monitoring helped maintain focus and allowed agile adjustments to address unforeseen delays or integration challenges effectively.

To visualize progress, a detailed Gantt chart was created, offering a clear, color-coded representation of all activities, their durations, dependencies, and responsible members. The chart encompassed every key deliverable, including literature review, dataset preparation, clause risk modeling, case outcome prediction, legal timeline visualization, and final report preparation.

This Gantt chart represents the detailed project schedule for "LawGrid: An AI Legal Assistant for Analogy-Driven Legal Reasoning and Support." It is divided into two main phases Project Conception and Initiation and Project Design and Implementation showing each task's start and end dates, responsible team members, and completion status. In the first phase (July–August 2025), activities include Group formation and topic finalization (handled by Ayush Gupta, Priyal Madvi, Sneha Gupta, and Rushikesh Palekar from 15 July 2025 to 15 July 2025), followed by discussion of the project topic using a paper prototype from 16 July to 3 August 2025 by the same members. Next, the team worked on identifying the functionalities of the major project between 5 August and 14 August 2025, and designing the Graphical User Interface (GUI) from 18 August to 24 August 2025 by Priyal Madvi and Sneha Gupta. The phase concludes with Presentation 1 on 25 August 2025, led by all four members.

The second phase (September–October 2025) emphasizes model development and implementation. Core model development and training were conducted by Ayush Gupta, Priyal Madvi, Sneha Gupta, and Rushikesh Palekar from 8 September to 22 September 2025. Then, Ayush Gupta and Sneha Gupta worked on developing and testing the system between 9 September and 22 September 2025, followed by integration of all modules and report writing from 25 September to 5 October 2025 by Priyal Madvi and Rushikesh Palekar. The second presentation took place on 6 October 2025. The final tasks include incorporating suggestions and refinements from 10 October to 17 October 2025 and submission of signed copies to the coordinator on 17 October 2025, with all team members participating. This time-

line clearly reflects a structured workflow, ensuring timely completion of each stage under assigned responsibilities.

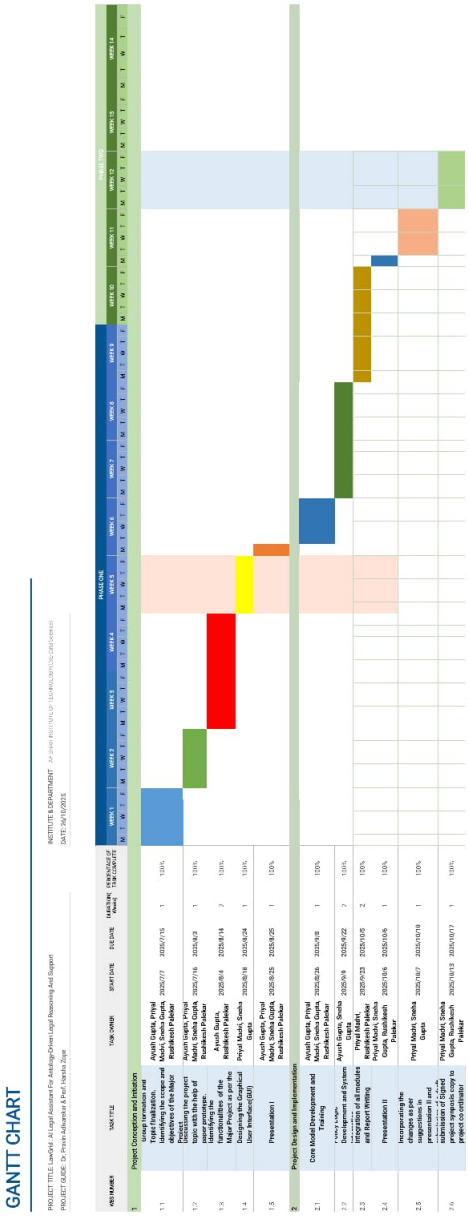

Figure 4.17: Timeline of the Project Milestone

Chapter 5

Testing

The testing environment for the AI Legal Assistant (AILA) was designed to ensure thorough validation of its core functionalities, including contract generation, risk detection, chatbot interaction, and legal event timeline creation. The backend, developed using Python and Natural Language Processing (NLP) models Core functionalities such as automated contract drafting, risk identification with plain-language explanations, and timeline generation were validated through real-world contract samples. The testing phase primarily focused on ensuring smooth communication, reliable NLP-based inference, real-time chatbot responsiveness, and user-friendly visualization of legal timelines across different simulated scenarios

5.1 Software Testing

The testing phase of the AI Legal Assistant system was crucial to ensure that all core modules functioned reliably under real-world legal scenarios. The system includes automated contract generation using NLP, risk detection powered by machine learning, a conversational chatbot for real-time assistance, and AI-driven legal event timeline creation. Software testing was executed across multiple layers from unit-level model validation to system-level workflows to verify that the platform responded accurately to inputs such as user-provided contract data, uploaded documents with varying complexity, and real-time user queries.

Particular attention was given to edge cases, such as incomplete or ambiguous contract input, detection of conflicting or duplicate clauses, inaccurate date recognition in legal documents, and chatbot misinterpretation of user queries. Both automated and manual tests were conducted to validate not only the core NLP and machine learning models but also the system's ability to provide accurate risk explanations and actionable recommendations dynamically a key requirement for a reliable and intelligent legal assistant.

Furthermore, performance testing evaluated the responsiveness of API calls for contract drafting and risk detection, ensuring real-time communication between backend modules and the frontend interface. Usability testing was also performed to assess the clarity of chatbot explanations, ease of navigation in the interface, and the readability of generated legal timelines. Compatibility tests were conducted across multiple browsers and screen resolutions, confirming stable operation even under resource constrained environments.

5.1.1 Testing Objectives

The overall testing outcomes validated the reliability, accuracy, and readiness of the AI Legal Assistant system for real-world deployment.

- Verify the modules such as contract generation, risk detection, chatbot, and timeline.

  smooth integration between backend, ML models, and frontend.

- Validate functional accuracy, performance, and responsiveness.

- Detect and fix bugs, NLP errors, UI issues, and latency problems.

5.1.2 Testing Methodology

A hybrid testing approach was adopted, combining structured documentation of test cases with detailed analysis of critical modules. Each test case included the module name, description, steps, expected vs. actual outcome, and pass/fail status to ensure systematic validation.

- Unit Testing: Performed on individual modules such as contract generation, risk detection, chatbot responses, and timeline extraction.

- Integration Testing: Validated interactions between modules, e.g., contract upload → risk detection → solution recommendation → timeline generation.

- Functional Testing: Ensured the system behaved correctly for real-world use cases, such as identifying ambiguous clauses, generating contracts, and responding to chatbot queries.

- Compatibility UI Testing: Conducted across different browsers, devices, and screen resolutions to verify consistent performance, responsiveness, and user experience.

5.1.3 Test Case Summary

The functional testing comprised 24 test cases across 20 core modules. Each test was executed using real-world legal documents and simulated user interactions. Test cases were categorized into UI testing, backend API validation, NLP accuracy, chatbot responsiveness, and legal timeline generation.

The test suite ensured comprehensive coverage of both typical and edge-case scenarios, including incomplete contract inputs, ambiguous clauses, incorrect date extraction, and unexpected chatbot queries. Results from these tests directly informed iterative refinements, such as improving NLP risk detection accuracy, reducing API latency, enhancing chatbot clarity, and resolving UI inconsistencies for a smoother user experience.

5.1.4 Performance Testing Insights

Performance testing was tailored to assess system responsiveness, query handling speed, and module latency under realistic legal use cases:

- The NLP-based document analysis module (contract clause extraction and risk detection) processed standard agreements in 1.2–1.8 seconds, ensuring near real-time feedback.
- Timeline generation and deadline calculation modules exhibited an average inference time of 900ms, even with multi-page legal documents.
- The chatbot interface responded within 1–1.3 seconds for typical legal queries, maintaining smooth conversational flow during consecutive interactions.
- UI transitions (e.g., Dashboard → Document Upload → Case Timeline) averaged below 800ms, tested under both Wi-Fi and 4G environments.

5.1.5 Security Testing

Given the sensitive nature of legal documents and client data, security testing was conducted to ensure confidentiality, integrity, and controlled access. Key checks included:

- Authentication Flows: Verified that only registered users could access personalized legal modules and that unauthorized access attempts were correctly blocked.
- Data Encryption: Confirmed that all uploaded documents and API calls were secured with encryption to prevent data leaks during transmission.

No critical security vulnerabilities were detected during testing. All mechanisms for authentication, encryption, and access validation worked as intended, ensuring secure handling of sensitive legal information.

5.1.6 Overall Outcome

All modules successfully passed their respective tests. Minor issues such as UI alignment in the chatbot interface and slight delays in document upload validation were resolved in later iterations. The testing confirmed that:

- Contract review and risk detection modules operate accurately with consistent results.
- The conversational chatbot responds effectively, providing clear legal explanations.
- Timeline extraction and event arrangement to ensure chronological accuracy.

5.2 Functional Testing

Functional testing was conducted to evaluate the performance and reliability of each core feature of the LawGrid AI system namely the Legal Chatbot, Event Timeline Scheduler, Document Generation, and Risk Analysis modules. The objective of this testing is to ensure that every module performs its intended function accurately and interacts seamlessly with the overall system architecture. Each componwent as tested under different scenarios to verify correct data processing, user interaction flow, and expected output generation.

During the testing process, each component was evaluated independently and in conjunction with the integrated system to verify smooth interaction between modules. The Legal Chatbot was tested for its ability to interpret user queries accurately and provide contextually relevant legal responses. The Event Timeline Scheduler underwent testing to confirm its precision in capturing, organizing, and updating case-related timelines. Similarly, the Document Generation module was validated for proper formatting, data extraction, and automated report creation, while the Risk Analysis component was tested to ensure it accurately identified potential legal risks based on input data and predefined parameters.

Overall, the functional testing phase ensured that LawGrid AI's modules not only performed their individual functions efficiently but also worked cohesively within the larger system architecture. This process helped identify and resolve integration issues, optimize workflow consistency, and guarantee that the platform delivers a seamless and reliable user experience in real-time legal operations.

TC ID	Module	Test Case Description	Expected Result	Status / Remarks
TC01	Chat Interface	Test if chatbot interface loads and accepts queries	Chatbox opens successfully and accepts text input	Pass
TC02	Legal Query Understanding	Verify chatbot understands user's legal questions using NLP	Chatbot correctly interprets questions (e.g., about IPC sections, rights, etc.)	Pass
TC03	Law Retrieval	Check chatbot retrieves relevant legal sections from dataset (e.g., Indian Kanoon / IndiaCode)	Accurate section and act displayed with explanation	Pass
TC04	Answer Generation	Validate LLM-generated response relevance and clarity	Contextual and human-like response generated	Pass
TC05	API Integration	Test backend Flask API and LLM communication	Stable response; no delay or crash during query processing	Pass
TC06	Dataset Access	Ensure chatbot can access and search legal dataset correctly	Dataset accessed, no missing or incorrect retrieval	Pass
TC07	Document Upload	Verify successful upload of multiple, supported file types (PDF, TXT, DOCX).	All files appear in the Uploaded Files list with a success message.	Pass
TC08	NL Processing	Verify the core logic extracts key legal dates and events from the document content.	Schedule shows relevant events (Hearings, Filings, Deadlines) and ignores irrelevant data.	Pass
TC09	chronological sorting	Verify the chronological sorting and source linking in the schedule.	Events are displayed in correct ascending date order, and each event clearly links to its source document.	Pass
TC010	Output Display	Verify the Refresh and Download/Export functions	Refresh updates the schedule. Download generates a correct, usable file of the current schedule.	Pass
TC011	File Management	Verify the Delete function on uploaded files and related records.	The document is removed successfully, and all associated events are cleared from the schedule.	Pass
TC12	Document Identification	Verify whether the system correctly identifies the appropriate legal document type based on the user's natural language problem description.	The system accurately identifies the relevant legal document (e.g., Legal Notice, Agreement, Affidavit) based on the input query.	Pass
TC13	Document Drafting (Live Edit)	Verify that the system generates a legal document draft dynamically based on user-provided inputs and updates the content in real time.	The legal document is generated in proper legal format and reflects real-time updates as the user edits input fields.	Pass

Table 5.1: Functional Testing Table for LawGrid AI

Chapter 6

Result and Discussions

The table 6.1 illustrates that the LLMs outperform older models because law is language-heavy and requires understanding and explanation, not just lookup. A legal chatbot must simplify complex Acts/judgments for users, something only LLMs can do well.

Criterion	**L**LM (Gemini 3 Flash)
Model Type	Large Language Model
Context Understanding	High
Generative Capability	Yes
Risk of Hallucination	Moderate
Legal Disclaimer Support	Required
Suitability for Legal QA	High

Table 6.1: Model Selection for Document QA System

Table 6.2 tells that the chatbot achieves a Relevance score of 4.5 out of 5 because it provides answers which match both user questions and the correct legal framework. The Completeness score of 4.2 out of 5 shows that answers contain all vital legal elements without deleting any important details. These help users grasp basic legal concepts.The system achieves a Clarity score of 4.6 out of 5 because it presents complex legal concepts through clear explanations which enable non-expert users to understand the system.

Metric	**Average Score**
Relevance	4.5 / 5
Completeness	4.2 / 5
Clarity	4.6 / 5
Disclaimer Presence	100%

Table 6.2: Evaluation of Chatbot Responses for Document Domain

The figure 6.1 dot plot compares three AI models—GPT-4-Turbo, LLaMA-3, and Gemini Flash 3—based on faithfulness, answer relevance, and response time. GPT-4-Turbo performs

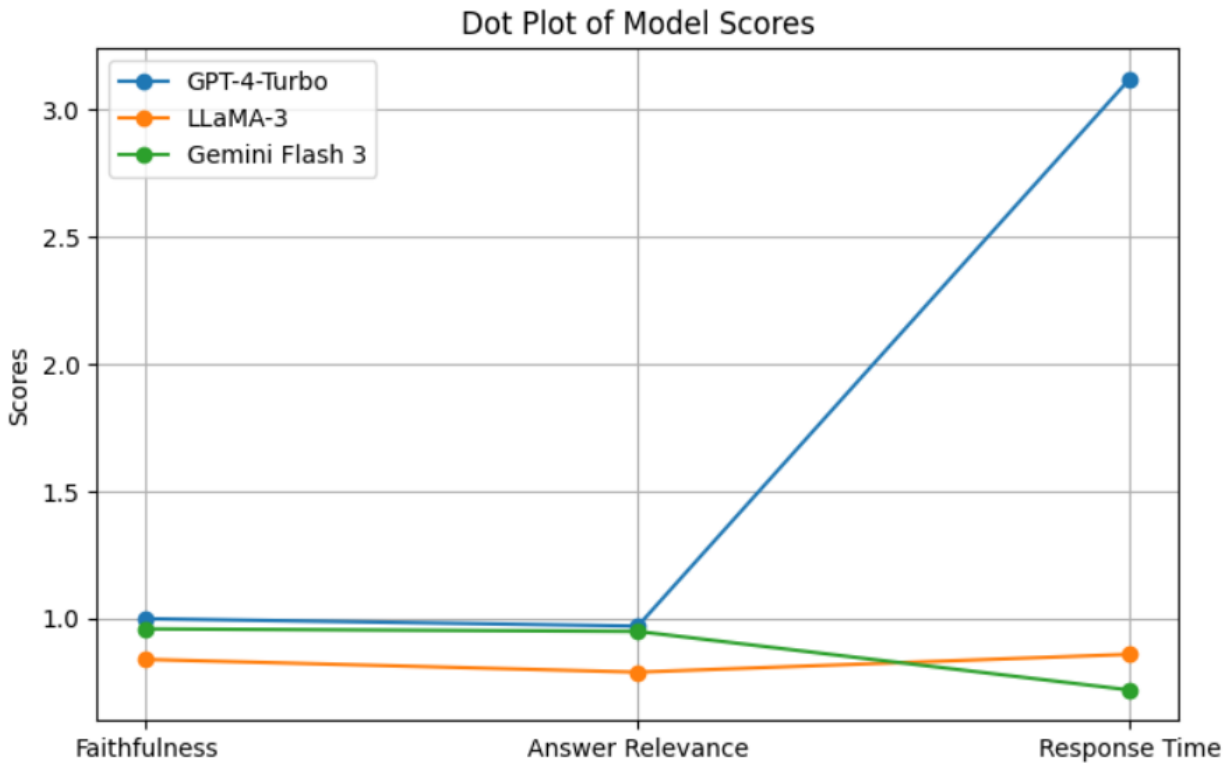

Figure 6.1: Comparison of Gemini Flash 3, GPT-4 turbo and llama 3 across evaluation metrics

the best in terms of both faithfulness and answer relevance, showing that it generates more accurate and contextually appropriate responses compared to the other models. Gemini Flash 3 comes very close to GPT-4-Turbo in these two quality metrics, indicating strong performance, while LLaMA-3 scores lower in both areas. However, when it comes to response time, Gemini Flash 3 clearly outperforms the others by being the fastest, followed by LLaMA-3, while GPT-4-Turbo is significantly slower. Overall, GPT-4-Turbo is the best choice for high-quality and reliable answers, whereas Gemini Flash 3 is more suitable when faster responses are required, and LLaMA-3 provides moderate performance across all metrics.

The table 6.3 illustrates that our proposed system focuses on extracting and analyzing legal information Using tools like pdfplumber and docx2txt, the text is accurately extracted from PDF and Word files for further processing. The extracted content is then processed through Named Entity Recognition (NER) using spaCy and regular expressions to identify key entities such as and dates. Finally, event chronology is established using NLP and date parsing techniques to arrange legal events in a sequential timeline.

Objective	**Implemented Feature**	**Output**
Legal Text Extraction	pdfplumber,docx2txt, pytesseract	pdfplumber,docx2txt
Named Entity Recognition	spaCy + Regex	Identified legal entities, organizations, and dates
Event Chronology	NLP + datetime	Sorted events by their date

Table 6.3: Result Table for Event timeline Legal Document

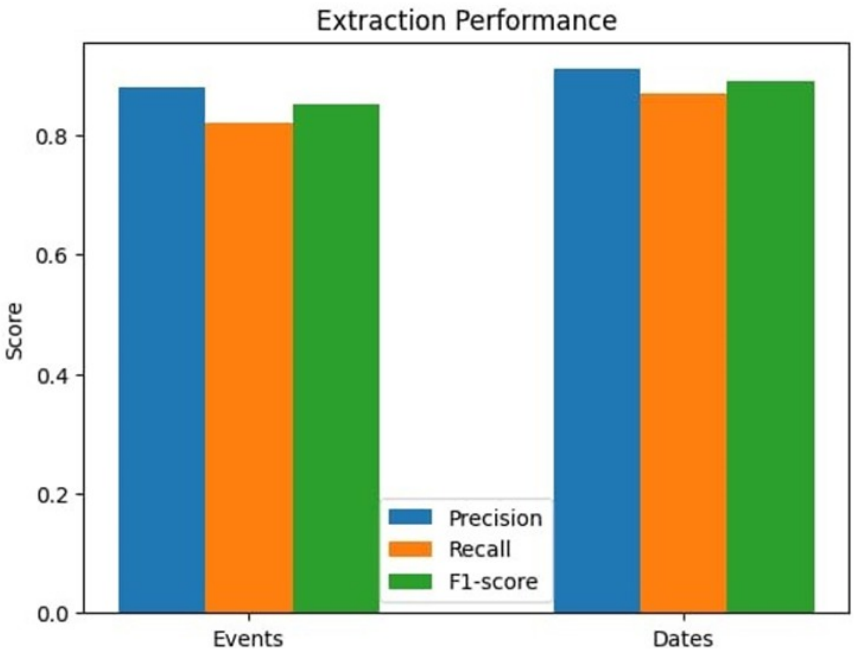

Figure 6.2: Extraction Performance

The figure 6.2 bar chart compares precision, recall, and F1-score for event and date extraction. Date extraction achieves the highest performance, indicating accurate temporal data extraction.

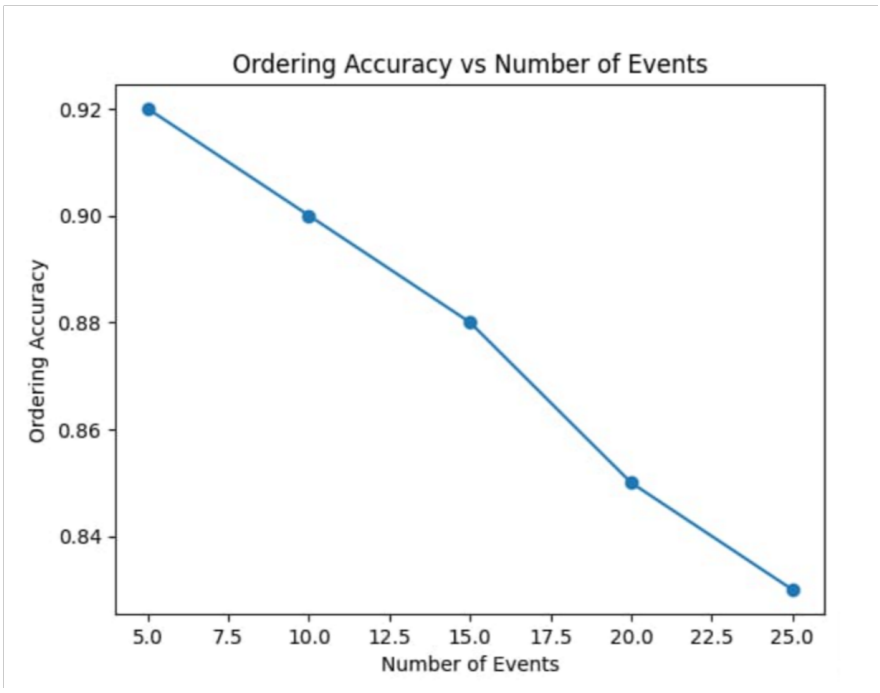

Figure 6.3: Ordering Accuracy vs Number of Events

The figure 6.3 illustrates that chronological ordering accuracy is highest for a smaller number of events and decreases gradually as the number of events increases.

Parameter	**Manual Review**	**AI Assisted System**
Time Requirement	High	Lower
Human Effort	High	Reduced
Risk of Missing Clauses	Moderate	Lower
Explanation Clarity	Variable	Consistent
Suggestions for Mitigation	Manual	Automated
Consistency Across Reviews	Reviewer-dependent	System-driven

Table 6.4: Qualitative Comparison of Manual and Automated Document Analysis

The Table 6.4 shows a qualitative comparison between conventional manual legal evaluation and the AI Assisted system, which operates under the new proposed system. The manual evaluation process demands employees to spend their time while they need to perform their work manually. The quality of results depends on how skilled the reviewer is, which creates differences in the final results. The automated system reduces the time needed for review because it performs automated clause analysis. This produces consistent results. The automated system protects vital clauses through its established evaluation standards. It performs systematic anal- ysis to prevent any vital clauses from going undetected. System provides users with better explanation clarity through its automated threat response system, which also improves system usability and interpretability. The comparison shows that AI-assisted legal analysis systems provide better decision support when handling large volumes of cases, which require standardized processing and quick execution, but need human legal validation.The comparison demonstrates how AI-assisted legal analysis functions as a decision support system which provides benefits when organizations need to handle large cases with exactness, and achieve high operational efficiency. Legal professionals need to verify the results. AI systems provide through their final validation process.

Chapter 7

Conclusion

The LawGrid AI Legal Assistant For Antology-Driven Legal Reasoning And Support project represents a transformative step in the integration of artificial intelligence within the legal domain. By combining natural language processing (NLP), machine learning, and intelligent automation, the system addresses critical challenges in the legal sector, such as the lack of personalized insights, time-consuming manual contract reviews, and limited accessibility to affordable legal support. This project demonstrates how technology can be effectively applied to simplify complex legal tasks, making legal services more transparent, efficient, and user-friendly.

At the core of the system lies its ability to review and generate contracts, identify potential risks, and provide clear explanations with practical solutions. Unlike traditional tools that only offer generic templates or require costly manual review, this AI-driven approach ensures customized outputs based on individual user input. By analyzing uploaded legal documents, the system highlights clauses that may present legal or compliance risks and explains them in plain, understandable language. This not only enhances user awareness but also empowers individuals and businesses to make informed legal decisions with confidence.

In addition, the project incorporates advanced features such as a conversational chatbot for interactive legal guidance and a timeline generator to arrange important legal events and deadlines automatically. These innovations make legal processes more accessible by enabling natural communication and simplifying complex information into structured formats. Overall, the AI Legal Assistant project highlights the potential of AI to revolutionize the legal field by improving accuracy, accessibility, and trust in legal services, ultimately bridging the gap between legal expertise and user understanding.

Chapter 8

Future Scope

The future scope of this project is highly promising, as AI and NLP continue to transform the legal domain. While the current system focuses on simplifying legal documents, generating timelines, and providing AI-powered assistance, it can be significantly enhanced in the future. By integrating with larger and more comprehensive legal databases, including case laws, regulations, and international contracts, the system can offer more accurate and wide-ranging answers. Advanced risk analysis could allow the AI to predict potential legal issues, suggest preventive measures, and simulate outcomes of contractual decisions. Adding multilingual support would make the system accessible to international clients and businesses, while voice-based interaction and smart assistant integration could make legal assistance even more convenient. The AI could also evolve to assist in automated contract negotiation, suggesting modifications and highlighting important clauses for discussion. Further integration with workflow and project management tools can streamline tracking of deadlines, approvals, and legal obligations. By learning from user feedback, the system can continuously improve its accuracy and relevance, and future developments could even incorporate blockchain-based smart contracts for secure, automated legal processes. Overall, the project has the potential to grow into a fully intelligent legal assistant, making legal processes faster, more transparent, and accessible to everyone from individuals to small businesses and large organizations.

Bibliography

[1] Youssra Amazou, Faouzi, Tayalati, Houssam, Mensouri, Abdellah, Azmani, Monir Az mani, "Accurate AI Assistance in Contract Law Using Retrieval-Augmented Generation to Advance Legal Technology", Vol 16, No 2, 2025, ResearchGate.

[2] P. Vimala Imogen, J. Sreenidhi, V. Nivedha, "AI-Powered Legal Documentation Assistant", ISSN: 2582-2012, 2024, ResearchGate.

[3] Manikrao Dhore, Aryan Vimal, Avish Agrawal, Raghav Bajaj, Rohan Barde, "Bettercall: AI based legal assistant", 2024, International Conference on Image Processing and Capsule Networks (ICIPCN).

[4] Kuldeep Vayadande, Aditya Bhoyar, Aditi Bhat, Zulfikar Charoliya, Pranav Bach hav, Aayush Chavan, "AI-Powered Legal Documentation Assistant", 2024, International Conference on Pervasive Computing and Social Networking (ICPCSN).

[5] Allan Almeida, Shubh Shah, Neer Bhavesh Khandor, Neha Agarwal, "LEGAL AI: A Tool to help Lawyers Optimize their Performance", 2024, IEEE.

[6] Sweta Priyadarshni, "AI-Driven Document Automation and Compliance in Contract Life cycle Management", 2024, IEEE.

[7] Divya Mehta, Tanvi Malhotra, Arpan Kumar and P.K. Jain, "An Approach to Get Legal Assistance Using Artificial Intelligence", 2023, IEEE.

[8] Dr. Hemlata Sharma and Aakanksha, Artificial Intelligence and Law: An Effective and Efficient Instrument, 2020, IEEE.

[9] Dipankar Chakrabarti, Neelam Patodia, Udayan Bhattacharya, Indranil Mitra, Satyaki Roy, Jayanta Mandi, Nandini Roy, Prasun Nandy, "Use of Artificial Intelligence to Analyse Risk in Legal Documents for a Better Decision Support", 2018, IEEE.

[10] Ashish Sangale, Mrunal Nalawade, Ameya Brahmankar, Shruti Agrawal, "Event Notifier on Scraper Mails using NLP",2020, IRJET.

[11] Qudsiya Naaz, Afifa Harmain, Ahmed Umme Aiman, Shital Bagde, Javeriya Khan, "NLP Enhanced Smart Calendar", May 2025, International Journal on Advanced Computer Theory and Engineering.

[12] Pavithiran G, Sharan Padmanabhan, Ashwin Kumar BR, Vetriselvi A, "Social Media Personal Event Notifier using NLP and Machine Learning", October 2022, Research Gate.